Verification-Driven AI Engineering: Workflows and Reference Architecture

Qinghua Lu[1], Liming Zhu[1], Suyu Ma[1], Helen Power[1], and Robert Speight[1]
[1] *CSIRO, Australia*

Abstract

Agentic AI, spanning agentic coding, agentic co-work, and agentic co-scientist settings, has shifted the verification scalability demands of AI engineering workflows. Candidate code and scientific solutions can now be generated at massive scale, which requires verification to be automated and scalable as much as possible, leaving only a small set of unavoidable parts for non-scalable verification, such as human judgement or wet-lab experiments. However, many coding and scientific problems are still decomposed in traditional ways that mix scalable verification parts with unscalable parts, forcing whole results into unscalable verification. This article presents a verification-driven AI engineering approach, including a workflow model and a reference architecture, to address this problem. The workflow model's design-time decomposition reformulates problems in a way that maximises scalable verification and stackable improvements with explicit acceptance rules, while runtime execution autonomously follows that plan and routes only a smaller amount of inherently non-scalable verification. The reference architecture operationalises this model through design patterns such as solver-verifier separation and context-aware verification routing. We evaluate the usefulness of the approach through two scientific AI use cases, PhosMine and SYNAPS, and show how it increases scalable verification coverage while isolating unavoidable non-scalable judgement.

Introduction

AI engineering is the application of software engineering principles and techniques to the design, development, and operation of AI systems [1]. This engineering practice is now being reshaped by agentic AI across agentic coding, agentic co-work, and agentic co-scientist settings: systems can generate candidate code and scientific solutions at massive scale, but verification often fails to scale and remains fragmented, expensive, and human-dependent. A central engineering question therefore becomes how to make verification as automated and scalable as possible, leaving only a small set of unavoidable parts for non-scalable verification, such as human judgement or wet-lab experiments [2].

Emerging work on verification-driven AI engineering is still limited but growing. Existing studies span different concerns, such as verification-oriented decomposition and workflow design [3], as well as inference-time verification dynamics [4]. However, many coding and scientific problems are still decomposed in ways that mix scalable and non-scalable verification in the same units of work, which forces whole outputs into expensive non-scalable review. This article builds on the emerging line of work by combining workflow and architectural perspectives in a unified engineering approach.

In this article, we propose verification-driven AI engineering as a new paradigm in AI engineering, in which verifiability is treated as a first-class design objective, where problems are deliberately formulated and decomposed to enable scalable and automated verification with occasional human involvement. The core idea is to formulate and decompose problems around verifiability, so that verification logic shapes both how solutions are generated and how they

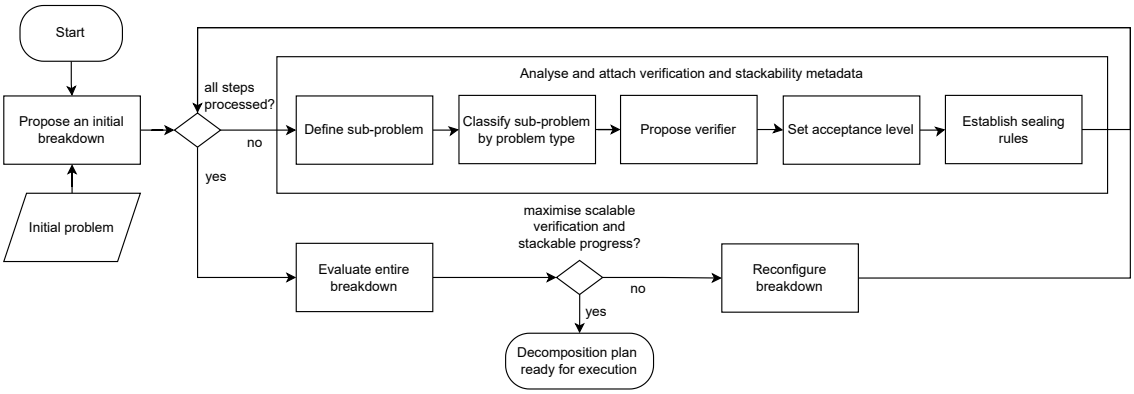


Figure 1: Design-time problem formulation and decomposition into verifiable, stackable sub-steps.

are accepted, sealed, or reopened. The approach combines two aligned artefacts: a case-level workflow model for design-time problem decomposition and runtime workflow execution, and a system-level pattern-based reference architecture that operationalises the workflow model through architectural patterns such as solver-verifier separation and context-aware verification routing. The workflow model defines verification surfaces, acceptance criteria, and stackability rules at step level, while the reference architecture provides reusable structural guidance for implementing these controls in practice. We evaluate the approach using two scientific AI use cases, PhosMine and SYNAPS, showing increased scalable verification coverage while isolating unavoidable non-scalable judgement.

CASE-LEVEL WORKFLOW MODEL

The case-level workflow model is defined by two workflows: *design-time problem decomposition* and *runtime execution*.

Design-Time Problem Decomposition Workflow.

As shown in Figure 1, the design-time workflow focuses on how a problem is formulated and decomposed into a set of verifiable, stackable steps that can be executed at runtime. Starting from an initial problem statement, the system proposes a candidate decomposition which produces an ordered list of candidate steps. Each step corresponds to a sub-problem that produces an artefact intended for downstream use or composition.

Each sub-problem undergoes a structured design-time analysis to elicit the verification and stackability information explicitly. First, the sub-problem is defined in terms of what is being solved and the artefact it is expected to produce. Second, the sub-problem is classified using a lightweight problem-type tag (e.g., checkable, context-judgement, or open-ended). This classification guides the selection of appropriate verifiers and sealing strategies. Third, a candidate verifier is proposed for the sub-problem. The verifier specifies how the produced artefact will be judged, for example using automated AI tools, mathematical tools, simulation tools, or human evaluation. The proposed verifier is classified according to its scalability, distinguishing between scalable (automated) verifiers and non-scalable (human or resource-intensive) verifiers. Fourth, an acceptance level is specified. For sub-problems that admit definitive correctness, the acceptance level is defined as pass/fail. For sub-problems that do not admit a single correct outcome, a "good enough" or threshold-based acceptance criterion is established. Acceptance levels are calibrated to balance error correction and false rejection during iterative solve-verify cycles [4]. Finally, sealing rules are defined to determine the stackability of the step. This includes whether

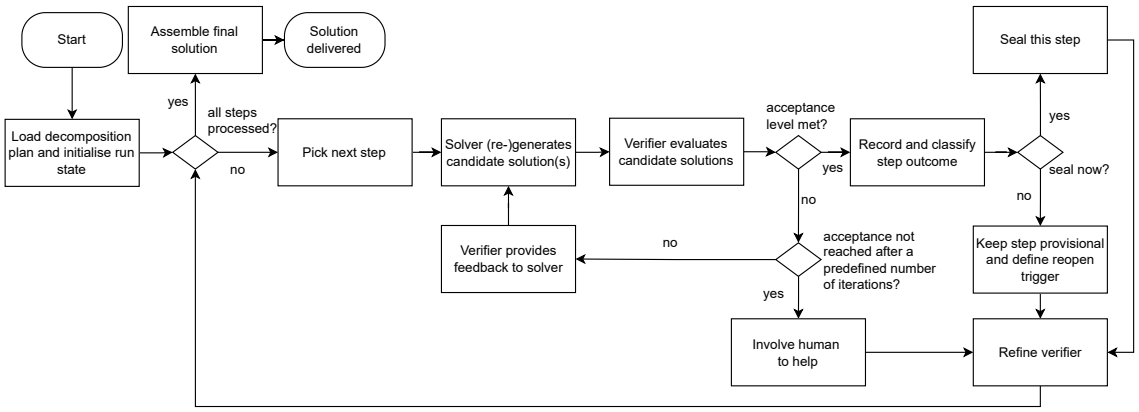


Figure 2: Autonomous runtime execution of the decomposition plan, with human involvement mainly for non-scalable verification.

the step can be sealed or must remain provisional, together with explicit reopen triggers that specify the conditions under which the step should be revisited.

After these steps are completed, the full set of verification and stackability metadata is attached to the sub-problem as a first-class design element. For each sub-problem, this metadata includes:

- the sub-problem definition: what is being solved and what artefact it produces;
- the sub-problem type: checkable sub-problems typically support scalable, automated verification and are therefore strong candidates for sealing; context-judgement sub-problems generally require non-scalable human input and are consequently more likely to remain provisional; open-ended sub-problems that rarely admit definitive correctness and are instead evaluated using threshold-based or "good enough" acceptance criteria.
- the proposed verifier: how we will judge the artefact;
- the verifier class: scalable (automated) or non-scalable (human or costly);
- the acceptance level: definitive (pass/fail, correct/incorrect as judged by the verifier) or "good enough" (meets an adequacy threshold to proceed, not claiming a single correct answer exists);
- the sealing rule (stackability): "sealable" if later steps can safely build on it without reopening, "provisional" if later evidence is likely to force revision, "reopen triggers" (the specific conditions that would justify revisiting it).

The complete breakdown is then evaluated holistically against verification-first objectives, including

- Verification scalability: how many steps have scalable verifiers, and how strong they are;
- Stackability: how many steps are sealable, and how robust the reopen triggers are;
- Residual non-scalable load: where humans are required and how frequently.

The evaluation also considers dependency structure (for example, where provisional steps appear and how many later steps rely on them); if criteria are not met, the decomposition is revised iteratively. The output of this workflow is a *decomposition plan*, consisting of an ordered final set of sub-steps, each with an explicit verifier, acceptance level, and sealing rule. This plan serves as the structural input to the runtime workflow.

Runtime Execution Workflow

Fig. 2 illustrates the autonomous runtime execution of the decomposition plan, with human involvement mainly for non-scalable verification. Execution begins by loading the decomposition plan produced at design-time. The runtime system then initialises its run state, including a current step pointer, a context store (capturing facts, artefacts, and externally supplied information), a sealed artefact store (containing accepted and sealed step outputs), and a provisional artefact store (storing the accepted as good enough but revisitable step outputs).

At runtime, a step-selection loop advances through the plan or reopens earlier provisional steps when triggers fire. The system assembles inputs from sealed/provisional stores and context, autonomously acquiring missing context except where human input is explicitly required. Once the required inputs are available, the AI solver generates one or more candidate solutions for the current step. These candidates are then evaluated by executing the corresponding verifier. When verification is performed automatically, the verifier produces pass/fail outcomes or scored results together with supporting rationale or diagnostic judgements. When verification is delegated to a human verifier, the human verifier provides an accept/reject decision, accompanied where appropriate by a brief rationale or adequacy judgement. Verifier outcomes are checked against definitive or "good enough" acceptance rules. If a step fails, feedback is routed to the solver for local refinement until acceptance or retry limits, after which human intervention is triggered.

Once a candidate solution satisfies the acceptance criteria, the resulting artefact is recorded together with its verification evidence. The artefact is then classified as sealed or provisional according to its sealing rule, which determines its stackability and governs whether the step may be revisited during execution. A sealed step is considered as final and will never be reopened. A provisional step allows execution to proceed to the next step for now, but may be reopened later. Reopen triggers apply only to provisional steps and specify the conditions under which reopening is permitted. Execution traces (candidates, verifier rationales, acceptance decisions, reopen/rollback events) are logged and can be used to calibrate verifier policies between runs. After each accepted step, reopen triggers are checked; if triggered, the system rolls back to the minimal affected point and resumes execution.

The runtime workflow terminates when all steps in the decomposition plan have been accepted. At this point, the system assembles the final solution from sealed artefacts and any permitted provisional artefacts. If the plan includes an optional end-to-end verifier, it is executed as a final gate. Failure triggers the same reopen and rollback mechanism, while success completes execution. By default, execution is autonomous, with human involvement limited to explicitly designated non-scalable verification steps and any explicitly human-sourced context acquisition. Only if execution fails structurally, such as being unable to meet acceptance criteria after repeated refinement, or repeatedly failing a global verification gate, the system escalates beyond runtime execution to redesign or re-decomposition.

SYSTEM-LEVEL REFERENCE ARCHITECTURE

Fig. 3 illustrates the system-level verifiability-by-design reference architecture. The architecture is organised around a clear separation of concerns between verification and solution generation, and is designed to realise the design-time problem decomposition plan and runtime execution of the decomposition plan in a verification-driven manner. In the figure, white boxes represent architectural components, while grey boxes denote recurring architectural patterns that shape component design and interaction. These patterns were identified through a systematic literature review in our previous study [2]. An empirically grounded design methodology has been adopted for designing the reference architecture [5]. The type of our reference architecture is an industry-crosscutting, classical, facilitation reference architecture. Our design strategy is a

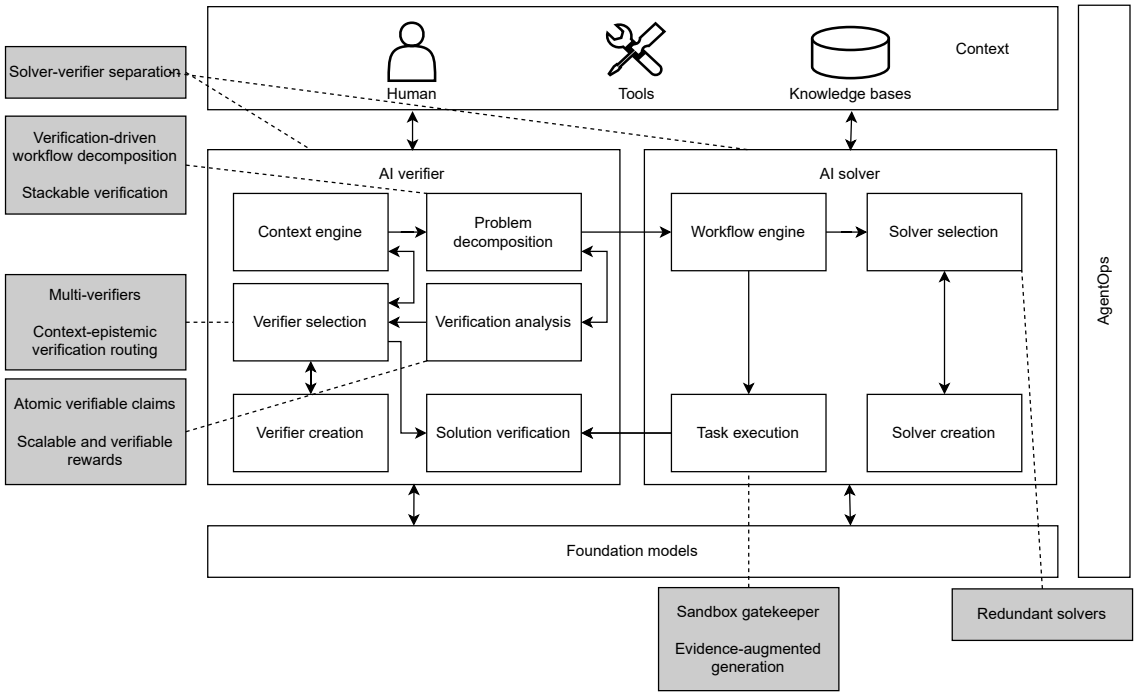


Figure 3: System-level verifiability-by-design reference architecture.

combination of research-driven and practice-driven, as the design of this reference architecture is founded mainly on the findings of systematic literature review [2] and our project experience [1].

At the top level, the system operates within a shared context comprising human input, external tools, and knowledge bases. Human actors play an explicit role in problem articulation, non-scalable verification, and context provision when required, but are not assumed to be in the execution loop by default. Tools and knowledge bases provide external capabilities and information that may be accessed by both solver and verifier components during execution.

The architecture is structured into two primary subsystems: the AI verifier and the AI solver, both grounded on underlying foundation models. This separation reflects the architectural pattern of solver–verifier separation, ensuring that solution generation and solution verification are decoupled both conceptually and operationally. Solvers do not self-verify the results and all progress is gated by verification outcomes.

AI Verifier Subsystem

The AI verifier is responsible for ensuring that verification remains a first-class concern. The problem decomposition component structures the problem into verifiable sub-steps, consistent with the design-time workflow. It realises the architectural pattern of verification-driven workflow decomposition, ensuring that execution structure is dictated by verifiability rather than solver convenience. It also reflects the stackable verification pattern, whereby sub-problems are deliberately shaped so that their outputs may be independently verified, sealed, and reused. Together with the verification analysis component, the problem decomposition component determines how each sub-problem should be verified and whether it admits scalable or non-scalable verification. This process operates over explicitly defined atomic claims, reflecting the atomic verifiable claims pattern, in which verification targets minimal, well-scoped units of assertion rather than opaque end-to-end outputs. By exposing clear verification signals at the level of individual sub-steps, this design enables the use of scalable and verifiable rewards, allowing

[1] https://www.csiro.au/en/research/technology-space/ai/science-digital

solver behaviour to be guided by verification outcomes rather than task completion alone. For example, instead of rewarding a solver only when an entire solution is produced, the solver can be incrementally rewarded when intermediate artefacts satisfy their corresponding verification criteria.

The solution verification component evaluates candidate solutions produced by the solver against the specified verification criteria and atomic claims. It produces explicit verification outcomes (such as pass/fail decisions, scores, or adequacy judgements) together with supporting evidence, which are then used to gate progress, trigger refinement, or determine sealing decisions. The verifier selection and verifier creation components manage the identification and instantiation of appropriate verifiers for each sub-problem. This supports the multi-verifiers pattern, enabling heterogeneous verification mechanisms to be applied according to the verification properties and acceptance requirements of each sub-problem. The context engine maintains assumptions, constraints, and externally supplied information that may affect verification outcomes. In conjunction with verifier selection, it enables context–epistemic verification routing, whereby verification strategies adapt dynamically based on available context, uncertainty, and epistemic state, rather than following a fixed or uniform verification pipeline.

AI Solver Subsystem

The AI solver subsystem is responsible for generating candidate solutions under the control of the verification-driven workflow. It includes components for workflow engine, solver selection, solver creation, and task execution. The workflow engine enforces the ordered execution of steps defined in the decomposition plan. Rather than allowing solvers to operate autonomously, it ensures that solver activity is coordinated according to verification requirements. This realises the solver–verifier separation pattern, in which solution generation is architecturally decoupled from solution verification and cannot advance without verifier approval. The solver selection and solver creation components allow different solution strategies or models to be employed for different sub-problems. This supports robustness and diversity in solution generation and aligns with the redundant solvers pattern, which enables cross-checking of independently generated solutions when appropriate. The task execution component invokes selected solvers to generate candidate solutions, which are then routed back to the verifier for evaluation. Solver behaviour may be constrained by sandbox gatekeeping, whereby solvers operate within controlled execution environments and their outputs are prevented from affecting the external system or real-world environment until verification is completed. Solver outputs may also be enhanced through evidence-augmented generation, which encourages the production of explicit justifications, references, or intermediate artefacts.

Use Cases

This section evaluates the usefulness of the approach through two scientific AI use cases.

Use Case A: PhosMine

PhosMine is an LLM-powered information extraction pipeline, which extracts experimentally supported protein-site-effect statements and their functional context from full-text articles at scale. In traditional phosphosite extraction workflows, quality control is typically added after model outputs are generated, rather than shaping how the task is decomposed from the outset. A typical workflow proceeds from text preprocessing to entity and relation extraction, followed by heuristic filtering or confidence-based ranking. When outputs appear unreliable, manual review is applied broadly to assess both trivial issues, such as formatting inconsistencies or identifier mismatches, and substantive biological interpretations. Because easily checkable aspects such as

document structure, evidence provenance, and identifier validity are mixed with interpretation-heavy reasoning about functional effects or pathway context, verification remains largely non-scalable. Human effort is therefore spread across both simple validation tasks and genuinely complex judgement calls, making it difficult to isolate errors, define stackable intermediate artefacts, or reuse partial results independently.

PhosMine adopts the verification-driven approach that separates automatically verifiable invariants from outputs whose verification requires higher-cost, threshold-based, or context-dependent judgement at design time. The workflow is organised around intermediate artefacts with explicit verification surfaces. Steps that admit strong machine-checkable interfaces, such as structural parsing, schema validation, provenance tracing, and identity normalisation, are isolated and can be sealed once accepted. Interpretation-sensitive outputs, including candidate substrates, consolidated protein-site effects, and ontology paths, are explicitly treated as provisional artefacts with potential reopen triggers. This restructuring confines scalable verification to well-defined artefacts and directs human attention only to those sub-problems where uncertainty is inherent rather than incidental. By decoupling checkable invariants from context-judgement outputs, PhosMine enables more scalable verification and more selective use of human attention, while accepting that some biological conclusions must remain provisional. This makes the trade-off between one-shot accuracy and scalable verifiability explicit, prioritising verifiable and stackable progress over unconstrained end-to-end extraction.

We decompose phosphosite knowledge extraction into sub-problems where each unit of work produces an artefact with an explicit verification surface, acceptance level, and sealing rule. The central design principle is to classify sub-problems according to their verification properties: whether they are primarily checkable, context-judgement, or open-ended, and whether their corresponding verifiers are scalable or non-scalable. Decomposition is therefore guided not only by data flow, but by the possibility of scalable verification, stackability, and isolation of uncertainty.

(1) Literature ingestion and text structuring. In conventional workflows, PDF parsing and text structuring are treated as preprocessing details. Failures or inconsistencies are often discovered only after extraction results appear unreliable, and the usual response is to rerun large parts of the pipeline, making error attribution coarse. In PhosMine, the key design decision is to identify which aspects of document processing are genuinely checkable and therefore strong candidates for scalable verification and sealing. Document structuring is treated as a deterministic transformation producing a Markdown artefact with a checkable verification surface: parsing success, structural completeness, and file consistency can be validated automatically. Since later biological interpretation is unlikely to invalidate this artefact, the corresponding sealing rule is *sealable*.

(2) Abstract retrieval and candidate substrate identification. Candidate proteins are often inferred implicitly during extraction, so incorrect candidates are only discovered late and require broad review of downstream outputs. The boundary between candidate generation and evidence-supported claims is blurred. In PhosMine, candidate substrates are treated explicitly as hypotheses rather than latent by-products of extraction. This creates a distinct artefact whose verification surface is weaker than that of structural parsing but still partially checkable through scalable verifiers such as deduplication, lexical plausibility, and optional ontology alignment. However, because later evidence may legitimately invalidate or refine the candidate list, the acceptance level is threshold-based and the sealing rule remains *provisional*, with downstream evidence serving as a stronger verifier and potential reopen trigger.

(3) Paragraph-level event extraction. Event extraction often produces protein-site-effect statements directly, and quality is managed through model confidence, heuristic filtering, or

spot-checking. Evidence provenance is often partial, making it difficult to reject only the incorrect part of an output without discarding the whole record. The design principle in PhosMine is to isolate checkable structural invariants from higher-cost semantic interpretation. The LLM output is constrained to a fixed schema and required to include verbatim evidence sentences. This yields an artefact with explicit verification surfaces over provenance and structure: JSON validity, required fields, site format patterns, closed effect labels, and paragraph-level traceability can all be verified using scalable verifiers. These aspects are therefore strong candidates for sealing. By contrast, the biological interpretation of the extracted event remains provisional. This decomposition makes explicit that what is being sealed at this stage is not the biological claim itself, but narrower atomic properties such as provenance and schema validity.

(4) Protein-site consolidation. Conventionally, consolidation is heuristic or model-driven and may rewrite or compress evidence. When the merged result is inconsistent, it is often unclear whether the error arose in extraction, merging, or normalisation, and debugging spans multiple stages. In PhosMine, consolidation is designed around solver-verifier separation rather than as an unconstrained merge. The LLM solver proposes candidate merged records, while scalable verifiers check whether verifiable invariants are preserved, including non-empty verbatim evidence and categorical fields constrained to a closed set. Deterministic repair handles structural violations, while later identity validation acts as an additional gate. The resulting artefact remains *provisional*, because even if its structural properties are accepted, later evidence or reinterpretation may reopen the biological content.

(5) Functional and process annotation. Ontology assignment is traditionally performed in one shot, producing labels that are difficult to audit and difficult to revise locally when evidence changes. Verification is therefore weakly structured and largely pushed to manual review. The key shift in PhosMine is to convert an open-ended classification task into a sequence of smaller sub-problems with bounded verification surfaces. At each ontology layer, the system must choose exactly one child node (or `UNKNOWN`) from an explicit candidate set. Structural correctness can be verified automatically through option membership and constrained output format, while semantic adequacy remains context-dependent. Accordingly, the verifier is only partially scalable, the acceptance level is effectively "good enough," and the ontology path remains *provisional* with reopen triggers when new evidence or ontology changes arise.

(6) Protein and site normalisation. Normalisation is often mixed with extraction outputs and handled through ad hoc string matching. Failures propagate silently, and inconsistencies are cleaned up late through manual effort. In PhosMine, normalisation is treated as a checkable sub-problem with a strong scalable verifier grounded in external biological resources. Protein names are mapped to canonical gene symbols and sites are validated against canonical sequences. Because mapping success or failure is machine-checkable and later biological interpretation is unlikely to invalidate canonical identity once established, the resulting artefact is a strong candidate for sealing.

Across these steps, caching keyed by paragraph text, term lists, and configuration supports reproducibility and enables targeted re-execution: when a verifier fails, only the affected artefact is regenerated rather than rerunning the entire workflow. This strengthens stackability and reduces unnecessary recomputation. Importantly, the redesign is not merely about adding verification gates after extraction, nor about increasing the number of stages. Rather, it reshapes the units of work so that checkable invariants and context-judgement outputs no longer co-exist within the same artefact. The primary shift is therefore structural separation, not procedural expansion.

In this use case, the sealing rule is defined not by whether a verifier is deterministic, but by whether downstream activities are unlikely to invalidate the accepted artefact. Thus, seal-

ability is tied to downstream independence rather than determinism alone. Document parsing and paragraph segmentation are sealable because later biological reasoning cannot alter their correctness. In contrast, candidate substrate identification, merged protein-site records, and ontology paths remain provisional, even when some of their properties pass scalable verification, because new evidence or reinterpretation can legitimately reopen them. What is sealed in intermediate biological steps are therefore narrow invariants such as provenance, schema validity, and identity mapping, rather than the biological claims themselves.

At runtime, LLM components are used only as solvers for candidate generation, including candidate terms, paragraph-level extractions, merged records, and ontology choices, while acceptance is determined by verifiers such as schema validators, provenance checks, closed-label constraints, set-membership checks for ontology decisions, and deterministic normalisation through external resources.

When a verifier fails, refinement is localised. The system discards invalid records, applies deterministic repair where appropriate, or keeps artefacts provisional for potential later reopening rather than allowing weak outputs to silently propagate. In this sense, PhosMine already reflects the runtime pattern in which progress is gated by verification outcomes. Importantly, the workflow prevents unconstrained generative outputs from being treated as final knowledge: no protein-site-function claim is allowed to propagate without explicit evidence provenance and successful identity validation.

Importantly, this decomposition also reduces the residual non-scalable load by confining human attention to a small number of explicitly identified failure modes. Human review is not required for sealed artefacts such as parsed text, paragraph boundaries, or normalised identities. Instead, attention is selectively directed to unresolved ontology paths, conflicting evidence for protein-site effects, or records that fail to normalise despite satisfying earlier structural checks. Verifiability-by-design does not seek to remove human judgement entirely, but to localise it to the sub-problems that remain inherently non-scalable because verification requires per-instance expert judgement or external validation.

Use Case B: SYNAPS

SYNAPS is a protein function annotation workflow, which is designed to address a structural bottleneck in computational biology: the vast majority of known protein sequences lack reliable functional annotations. While billions of protein sequences exist in large biological databases, only a very small fraction have experimentally validated functional labels. Most annotations are therefore computational predictions with varying degrees of uncertainty. SYNAPS integrates multiple sources of computational evidence, including sequence and structure similarity search, machine learning models based on protein language model embeddings, and LLM reasoning. These heterogeneous solvers generate candidate functional annotations together with supporting evidence.

In traditional protein annotation pipelines, quality control is typically appended after predictions are generated rather than shaping how the task is decomposed from the outset. A typical workflow proceeds from similarity search to label transfer, followed by machine-learning predictions and confidence-based ranking. When predictions appear biologically implausible, manual review is applied broadly, covering both trivial issues such as malformed EC numbers and more complex biological questions such as pathway plausibility or organism-level compatibility. Because easily checkable aspects such as ontology validity, metadata consistency, and label formatting are mixed with interpretation-heavy reasoning about biological function, verification becomes largely non-scalable. Human effort is therefore distributed across both simple validation tasks and genuinely complex biological judgement calls, making it difficult to localise errors or treat intermediate outputs as independently reusable artefacts.

From a verifiability-by-design perspective, SYNAPS can instead be interpreted as implicitly

embodying a verification-first decomposition. Rather than collapsing multiple predictive signals into a single answer, the workflow preserves independent solver outputs and introduces an explicit reasoning layer that evaluates biological plausibility. Deterministically verifiable properties are isolated and sealed early, while interpretation-sensitive outputs remain provisional and revisitable. This restructuring confines scalable verification to well-defined artefacts and directs human attention only to those sub-problems where uncertainty is inherent rather than incidental.

The SYNAPS protein annotation process can be decomposed into a sequence of sub-problems where each step produces an artefact with an explicit verification surface, acceptance level, and sealing rule.

(1) Biological Input Assembly. Traditionally, protein sequence, structural information, and species metadata are treated as background context, so inconsistencies or missing structure may only surface when downstream predictions look implausible. Here, the workflow starts by structuring these inputs as explicit artefacts. Protein sequence data, structural predictions, and organism metadata are validated independently with deterministic checks for data integrity and metadata consistency. Because downstream reasoning cannot invalidate factual input properties, these artefacts support strong scalable verification and are therefore sealable.

(2) Multi-Source Functional Prediction. SYNAPS generates candidate functional annotations using multiple solvers, including sequence and structure similarity search as well as machine learning models based on protein language model embeddings. A typical pipeline ranks similarity-based labels and ML predictions by confidence, even though those signals are not directly comparable. Sequence similarity methods rely on alignment coverage, percentage identity, and E-value, whereas ML models emit model-specific confidence scores. When similarity search yields no plausible match, the sequence may sit in a low-support or out-of-distribution region for the ML model, but that warning is often overlooked. In the verification-driven formulation, each solver emits a structured artefact containing candidate functional labels (such as EC numbers or localisation predictions), confidence signals, and provenance information. Deterministic validators then check structural invariants such as EC formatting and ontology membership. Structural validity can be sealed, while biological interpretation of predicted functions remains provisional.

(3) LLM-Based Biological Plausibility Reasoning. Without clear boundaries, LLMs may generate explanations or reformulate predictions in ways that are inadvertently treated as authoritative summaries. In SYNAPS, by contrast, the LLM is positioned as a semantic plausibility verifier rather than a primary solver. It evaluates predictions in biological context, identifies biologically impossible annotations (for example, nuclear localisation predicted for a bacterium), checks species compatibility, and compares competing functional hypotheses. Outputs are constrained to structured plausibility assessments rather than unconstrained text. Because plausibility reasoning depends on biological interpretation, these artefacts remain provisional.

(4) Cross-Signal Integration and Acceptance Gating. In many systems, final annotations are selected through heuristic merging or confidence ranking across models. In this design, integration becomes an explicit acceptance gate: agreement between independent solvers increases epistemic stability, while explicit contradictions trigger refinement or refusal-to-annotate conditions. Structural invariants remain sealed, while functional interpretations remain provisional and reopenable.

(5) Downstream Biological Integration. SYNAPS integrates predicted annotations into downstream systems such as metabolic modelling and antibiotic target identification. Conventionally, accepted predictions propagate to downstream models even when some are biologically implausible. Under the verification-driven workflow, only artefacts that satisfy verification gates are propagated into downstream modelling. Provisional annotations remain explicitly marked and can be reopened if inconsistencies are detected during modelling.

Across these stages, intermediate artefacts are structured so that verifiable invariants and interpretation-dependent reasoning do not coexist within the same unit of work. When verification fails, only the affected artefact is regenerated rather than rerunning the entire workflow. This supports stackability and reproducibility.

At runtime, independent solvers generate candidate functional annotations. Deterministic validators check structural constraints, while the LLM evaluates biological plausibility. Acceptance decisions are governed by explicit verification gates. When verification fails, refinement is localised to the affected artefact. Persistent contradictions or out-of-distribution predictions may trigger human review. Importantly, the workflow prevents unconstrained generative reasoning from being treated as final biological knowledge. No functional annotation is propagated to downstream modelling without passing structural validation and plausibility checks.

This decomposition confines human attention to explicitly identified uncertainty points, such as persistent cross-model contradictions or ambiguous biological contexts. Human review is not required for sealed artefacts such as validated metadata or ontology-conformant label formats. Verification-first therefore does not eliminate human judgement, but localises it to inherently interpretation-dependent sub-problems.

CONCLUSION

This article presented a verification-driven approach to agentic AI engineering in which verification is designed into the workflow and enforced by architecture. We contributed a case-level workflow model for design-time decomposition and runtime execution, and a system-level reference architecture that operationalises the workflow model through architectural patterns. Using two scientific use cases, we showed how the approach localises human judgement, improves reuse through sealing decisions, and makes uncertainty explicit through provisional artefacts and reopen rules.

References

[1] Len Bass, Qinghua Lu, Ingo Weber, and Liming Zhu. *Engineering AI systems: architecture and DevOps essentials*. Addison-Wesley Professional, 2025.

[2] Liming Zhu and Qinghua Lu. Verifiability-first ai engineering in the era of aiware: A conceptual framework, design principles, and architectural patterns for scalable verification. *Design Principles, and Architectural Patterns for Scalable Verification (January 07, 2026)*, 2026.

[3] Mert Yuksekgonul, Daniel Koceja, Xinhao Li, Federico Bianchi, Jed McCaleb, Xiaolong Wang, Jan Kautz, Yejin Choi, James Zou, Carlos Guestrin, et al. Learning to discover at test time. *arXiv preprint arXiv:2601.16175*, 2026.

[4] Yuxuan Wan, Tianqing Fang, Zaitang Li, Yintong Huo, Wenxuan Wang, Haitao Mi, Dong Yu, and Michael R Lyu. Inference-time scaling of verification: Self-evolving deep research agents via test-time rubric-guided verification. *arXiv preprint arXiv:2601.15808*, 2026.

[5] Matthias Galster and Paris Avgeriou. Empirically-grounded reference architectures: a proposal. In *Proceedings of the joint ACM SIGSOFT conference–QoSA and ACM SIGSOFT symposium–ISARCS on Quality of software architectures–QoSA and architecting critical systems–ISARCS*, pages 153–158, 2011.